03. Attention & Seq2Seq

Overview

Sequence-to-sequence models compress an input sequence into a context vector, then decode outputs step by step. Attention removes the bottleneck by letting each decoder step read all encoder states with learned weights. This module builds the additive scoring function, softmax alignment, and training/inference loops you will implement in the lab.
Lab: Lab 03.
Prior modules: Module 01, **Module 02**.

Contents

Module track

This module builds on **Module 01**, **Module 02**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

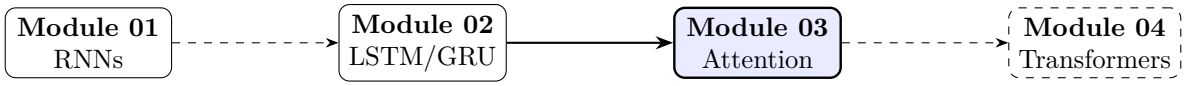

Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

Recap from Module 01 and Module 02

Encoder RNNs map source tokens to hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_T$. Using only $\mathbf{h}_T$ as context loses detail on long sequences; attention re-reads all encoder steps each decode step.

1 Encoder to Decoder Framework

Sequence-to-sequence models map variable-length inputs to variable-length outputs [2]. An encoder RNN (from **Module 01** (RNN recurrence)) produces context; a decoder RNN generates outputs autoregressively:

$$P(y_{1:T'} \mid x_{1:T}) = \prod_{t'=1}^{T'} P(y_{t'} \mid y_{<t'}, \mathbf{c}), \tag{1}$$

where $\mathbf{c}$ is a fixed context vector, often $\mathbf{c} = \mathbf{h}_T$, creating an information bottleneck.

2 Bahdanau Attention

Additive attention [1] replaces a single $\mathbf{c}$ with a time-dependent context:

$$e_{t',i} = \mathbf{v}^\top \tanh(\mathbf{W}_s \mathbf{s}_{t'-1} + \mathbf{W}_h \mathbf{h}_i), \tag{2}$$

$$\alpha_{t',i} = \frac{\exp(e_{t',i})}{\sum_{j=1}^{T} \exp(e_{t',j})}, \tag{3}$$

$$\mathbf{c}_{t'} = \sum_{i=1}^{T} \alpha_{t',i}\, \mathbf{h}_i. \tag{4}$$

Decoder state $\mathbf{s}_{t'}$ attends to all encoder states $\mathbf{h}_i$.

Attention as soft alignment. Large $\alpha_{t',i}$ means output step t' focuses on input position i, learned alignment without hand-crafted features.

3 Training Objective

Teacher forcing feeds ground-truth $y_{t'-1}$ during training; at inference the model consumes its own predictions. Loss is token-level cross-entropy summed over the target sequence:

$$\mathcal{L} = -\sum_{t'=1}^{T'} \log P(y^*_{t'} \mid y^*_{<t'}, x_{1:T}). \tag{5}$$

Beam search with width B retains top-B partial hypotheses at each step, approximating MAP decoding.

3.1 Exposure Bias

Mismatch between teacher-forced training and autoregressive inference causes error accumulation; scheduled sampling mitigates this.

4 Attention Variants Preview

Table 1: Attention scoring functions (see **Module 04** (scaled dot-product attention)).

Name	**Score** $e_{t',i}$
Dot-product	$\mathbf{s}_{t'}^\top \mathbf{h}_i$
Additive (Bahdanau)	$\mathbf{v}^\top \tanh(\mathbf{W}_s \mathbf{s}_{t'} + \mathbf{W}_h \mathbf{h}_i)$
Scaled dot-product	$(\mathbf{s}_{t'}^\top \mathbf{h}_i)/\sqrt{d_k}$

5 Visualization and Interpretability

Attention weights $\alpha_{t',i}$ form an $T' \times T$ matrix interpretable as alignment between source and target tokens.

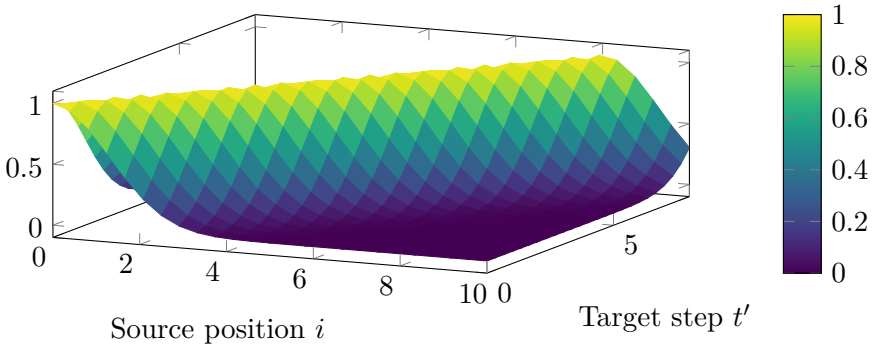

Figure 2: Example attention map (diagonal = monotonic alignment).

6 Connection to Transformers

Attention removes recurrence in the encoder when applied in parallel, the key step toward **Module 04** (self-attention). LSTM encoders in **Module 02** (LSTM gate equations) remain useful for streaming/low-latency settings.

7 Lab

Lab 03 implements additive attention on a toy translation task; visualize $\alpha_{t',i}$ from (3). Compare perplexity with and without attention against the bottleneck in **Module 01** (sequence output layouts).

8 Coverage and Length Penalties

Coverage penalty discourages attending to the same source positions repeatedly:

$$\text{cov}_i = \sum_{t'=1}^{T'} \alpha_{t',i}, \quad \mathcal{L}_{\text{cov}} = \sum_{t',i} \min(\alpha_{t',i}, \text{cov}_i). \tag{6}$$

Length normalization divides log-prob by $((5 + |y|)/6)^\alpha$ during beam search.

8.1 Luong Attention

Multiplicative score $e_{t',i} = \mathbf{s}_{t'}^\top \mathbf{h}_i$ is cheaper when dimensions align [1].

9 Extended Intuition

Encoder-decoder seq2seq compresses an input sentence into a single context vector $\mathbf{c}$, usually the final encoder hidden state. That bottleneck hurts on long inputs: early tokens get washed out by later ones, and the decoder cannot selectively retrieve information. Attention fixes this by letting each decoder step query all encoder outputs with a learned compatibility score.

Additive (Bahdanau) attention computes a scalar score per encoder position, softmaxes into weights $\alpha_{t',i}$, and forms a context vector as a weighted sum. The decoder RNN input at step t' becomes $[\mathbf{y}_{t'-1}; \mathbf{c}_{t'}]$, so generation is conditioned on both the previous token and the retrieved context. Training uses teacher forcing (feed gold previous tokens); inference feeds back model predictions, which introduces exposure bias.

Multiplicative (Luong) attention uses dot products and is cheaper when encoder and decoder hidden sizes match. Neither variant requires the encoder and decoder to share vocabulary or embedding tables, but hidden dimensions must align for dot-product variants. Transformers in **Module 04** generalize attention to self-attention over all positions, removing the RNN entirely.

9.1 Alignment as a Soft Pointer

Attention weights $\alpha_{t',i}$ form a probability distribution over source positions for each decoder step. Sharp peaks mean the decoder copies or paraphrases a specific source token; flat weights mean the model averages context and often produces generic output. Coverage penalty (optional extension) adds loss when $\sum_{t'} \alpha_{t',i} \gg 1$, discouraging repeated attention to the same source word (a common cause of repetition in output). Luong "general" attention uses $\mathbf{s}_t^\top \mathbf{W} \mathbf{h}_i$ when dimensions differ; always verify $\mathbf{W}$ is actually trained, not left at init.

10 Numerical Walkthrough

Toy translation: source length $T = 4$, target length $T' = 3$, encoder hidden $d_h = 64$, decoder hidden $d_s = 64$.

Encoder outputs $\mathbf{h}_1, \ldots, \mathbf{h}_4 \in \mathbb{R}^{64}$. At decoder step $t' = 2$, previous hidden $\mathbf{s}_1 \in \mathbb{R}^{64}$, previous token embedding $\mathbf{e}(y_1) \in \mathbb{R}^{32}$.

Additive score for source position i:

$$e_{2,i} = \mathbf{v}^\top \tanh(\mathbf{W}_1 \mathbf{s}_1 + \mathbf{W}_2 \mathbf{h}_i), \quad \alpha_{2,i} = \frac{\exp(e_{2,i})}{\sum_{j=1}^{4} \exp(e_{2,j})}. \tag{7}$$

If raw scores are $[0.5, 2.1, 0.3, -1.0]$, softmax gives $\alpha_{2,\cdot} \approx [0.13, 0.72, 0.11, 0.04]$: the model focuses on source token 2. Context $\mathbf{c}_2 = \sum_i \alpha_{2,i} \mathbf{h}_i$.

Batch $B = 16$, max source len 20, max target len 25: attention weight storage is $B \cdot T' \cdot T = 16 \cdot 25 \cdot 20 = 8000$ scalars per forward pass (negligible). Training with Adam, lr $= 10^{-3}$, on 100k sentence pairs might reach BLEU 18 to 22 on a small en-fr split after 10 epochs with $d_h = 256$.

Decoder init trick: set $\mathbf{s}_0 = \tanh(\mathbf{W}_s \overrightarrow{\mathbf{h}}_T + \mathbf{b}_s)$ instead of zeros so the first attention query already encodes the full source. Attention energy $e_{t',i}$ has scale tied to $\|\mathbf{v}\|$ and hidden norms; if energies drift to ± 50, softmax saturates and gradients vanish ("attention collapse"). Masking with $-\infty$ in fp16 can produce NaN; use `-1e4` for masked positions when training in half precision.

Batch matrix multiply for all decoder steps at once (if teacher forcing with fixed T'): stack decoder queries $\mathbf{S} \in \mathbb{R}^{B \times T' \times d_s}$, scores $\mathbf{SWH}^\top$ yield all α in one kernel. Memory for storing all α during training: $B \cdot T' \cdot T \cdot 4$ bytes; at $B = 32, T = T' = 50$ that is 320 KB, still small vs RNN activations. When BLEU stalls, inspect whether errors are alignment (wrong content word) or fluency (grammar); attention heatmaps distinguish the two failure modes.

11 PyTorch Implementation Notes

- Manual attention: store encoder outputs `(B, T,` d_h`)`; loop decoder steps or use `nn.GRUCell` with context concatenated to input.
- `nn.MultiheadAttention` expects `(T, B, E)` by default (`batch_first=False`); transpose if your pipeline uses batch-first tensors.
- Mask padding in encoder when computing scores: set $e_{t',i} = -\infty$ for pad positions before softmax.
- Teacher forcing probability 0.5 during training reduces exposure bias; decay toward 0 over epochs.
- `pack_padded_sequence` on the encoder; decoder still needs explicit length handling for loss masking.

```
# Luong dot attention (d_h == d_s)
scores = torch.bmm(dec_h.unsqueeze(1), enc_out.transpose(1, 2))
# scores: (B, 1, T_enc) -> softmax over T_enc
weights = F.softmax(scores, dim=-1)
context = torch.bmm(weights, enc_out).squeeze(1)  # (B, d_h)
```

12 Local GPU Constraints

Seq2seq with $d_h = 256$, $T, T' \leq 40$, $B = 32$ fits easily on 4 GB. Longer sequences ($T = 100+$) dominate memory through encoder/decoder unrolls; reduce B to 8 and use gradient accumulation.

Attention maps are cheap relative to RNN matmuls at these sizes. Profile before optimizing: on a 1650, encoder-decoder training is often CPU dataloader bound if you tokenize on the fly.

Copy attention (Gu et al.) adds a pointer to source tokens via α mixing over one-hot indices; rare in modern stacks but clarifies that α is a soft copy distribution. Length normalization during beam search divides accumulated log-prob by $(5 + |y|)^\alpha$ to penalize short outputs; without it beams favor EOS early. Precompute encoder outputs once per source sentence during inference; decoder-only loops reuse $\mathbf{h}_{1:T}$ across all beam hypotheses. Scheduled sampling: at step t', feed gold token with prob p and model token with $1 - p$; linearly decay p from 1.0 to 0.2 over training reduces exposure bias without cold-starting inference. Luong "concat" attention forms context from $[\mathbf{h}_i; \mathbf{s}_{t'}]$ before scoring; doubles effective width and needs projection back to d_h.

13 Debugging Checklist

- Decoder repeats one word: check SOS/EOS token ids, teacher forcing always on during inference, or empty attention from pad masking bugs.

- BLEU stuck near 0: reversed source/target by mistake, or decoder never sees encoder outputs (disconnected graph).
- NaN attention weights: softmax over all $-\infty$ rows when an entire source is pad; mask carefully.
- Train loss low, inference gibberish: classic exposure bias; add scheduled sampling or beam search at decode time.
- Shapes break on variable lengths: encoder packed but decoder assumes fixed T'.
- Attention entropy near $\log T$ every step: decoder ignoring encoder; check that `enc_out` requires grad and is connected to loss.
- Source reversed but target correct language: BLEU near zero despite falling loss; verify tokenization direction matches parallel corpus.

14 Practice Problems

Problem 1. For $T = 5$, write softmax weights if $e_i = [1, 1, 1, 1, 1]$. What does uniform attention imply about $\mathbf{c}$?

Problem 2. Implement additive attention with learnable $\mathbf{v}, \mathbf{W}_1, \mathbf{W}_2$ and verify $\sum_i \alpha_{t',i} = 1$.

Problem 3. Visualize $\alpha_{t',i}$ as a heatmap for 10 translated sentences. Which source tokens get highest weight at each step?

Problem 4. Compare greedy decoding vs beam search ($k = 5$) on 20 held-out sentences.

Problem 5. Compute per-step attention entropy $H_{t'} = -\sum_i \alpha_{t',i} \log \alpha_{t',i}$ for one sentence. Plot $H_{t'}$ vs t' and interpret peaks.

Problem 6. Ablate decoder init: zeros vs $\tanh(\mathbf{W}_s \overrightarrow{\mathbf{h}}_T)$. Compare BLEU after 5 epochs on a tiny parallel corpus.

Pointer-generator networks mix copy and generate distributions via λ from attention; inspect when $\lambda \to 1$ on entity-heavy sentences.

Build the full pipeline in **Lab 03**; self-attention in **Module 04** replaces this RNN encoder-decoder stack.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

[2] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *EMNLP*, 2014.

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[4] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. In *EMNLP*, 2015.

[5] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *NeurIPS*, 2014.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.